# 0.19. Revenir : un pas négatif

Le 0.18, plus une ligne : deplace_x avec -pas fait revenir la lettre. C'est là que le « x = » sert.

## 1. Ce qu'il faut comprendre

**C'est le 0.18, plus une ligne :** le retour.

**Ce qui est nouveau ici : un pas négatif.** `-pas`, c'est -5 : la lettre **recule** de 5 cases vers la gauche. Le signe décide du sens : positif vers la droite, négatif vers la gauche.

**Un octet ne connaît pourtant pas les nombres négatifs :** `-5` y est rangé comme 256 − 5 = **251**. `deplace_x` sait donc qu'au-delà de 127, c'est un recul.

**Le retour part de `x`.** Grâce au `x =` du 0.18, `x` vaut 5 : le retour part bien de la colonne 5, et rend 0. Sans le `x =`, en C ordinaire, `x` vaudrait encore 0 : le retour partirait du bord gauche, buterait dessus, et **ne ferait rien**. Le `x =` sert à **garder en mémoire où est la lettre**.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <deplace_x>` → 0.17.3. La fonction deplace_x() — avancer sur une ligne
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

### 📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

### 🔍 Le programme, ligne par ligne — chaque ligne dite en français

| | |
|---|---|
| `#include <deplace_x>  // fait avancer une tuile sur sa ligne` | Demande à la console « deplace_x » (ce que c'est : fait avancer une tuile sur sa ligne). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « fait avancer une tuile sur sa ligne ».) |
| `#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A` | Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».) |
| `uint8_t pas = 5;  // +5 : avance de 5 cases ; -5 : recule de 5 cases` | Une variable : une boîte appelée pas, qui retient un nombre de 0 à 255, et qui commence à 5. (Le commentaire le dit : « +5 : avance de 5 cases ; -5 : recule de 5 cases ».) |

| | |
|---|---|
| `uint8_t x = 0;  // la colonne de la lettre : elle part de 0 (tout à gauche)` | Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « la colonne de la lettre : elle part de 0 (tout à gauche) ».) |
| `int main() {` | Le début du programme : la console commence ici, par la première ligne qui suit. |
| `// L'ALLER (le 0.18) : 5 cases vers la droite ; x reçoit 5.` | Un commentaire : une phrase pour celui qui lit (« L'ALLER (le 0.18) : 5 cases vers la droite ; x reçoit 5. »). La console l'ignore. |
| `x = deplace_x(x, 0, ALPHABET[0], pas);  // après cette ligne : x vaut 5` | x prend une nouvelle valeur : deplace_x(x, 0, ALPHABET[0], pas). (Le commentaire le dit : « après cette ligne : x vaut 5 ».) |
| `// LE RETOUR, la ligne ajoutée :` | Un commentaire : une phrase pour celui qui lit (« LE RETOUR, la ligne ajoutée : »). La console l'ignore. |
| `//   x = deplace_x(x, 0, ALPHABET[0], -pas);` | Un commentaire : une phrase pour celui qui lit (« x = deplace_x(x, 0, ALPHABET[0], -pas); »). La console l'ignore. |
| `//                |                  |` | Un commentaire : une phrase pour celui qui lit (« \| \| »). La console l'ignore. |
| `//                |                  +-- -pas : -5, un pas NÉGATIF, donc vers` | Un commentaire : une phrase pour celui qui lit (« \| +-- -pas : -5, un pas NÉGATIF, donc vers »). La console l'ignore. |
| `//                |                  la gauche. (Dans l'octet, -5 est rangé` | Un commentaire : une phrase pour celui qui lit (« \| la gauche. (Dans l'octet, -5 est rangé »). La console l'ignore. |
| `//                |                  251 ; au-delà de 127, c'est un recul.)` | Un commentaire : une phrase pour celui qui lit (« \| 251 ; au-delà de 127, c'est un recul.) »). La console l'ignore. |
| `//                +-- x vaut 5 grâce à l'aller : le retour part de la colonne 5` | Un commentaire : une phrase pour celui qui lit (« +-- x vaut 5 grâce à l'aller : le retour part de la colonne 5 »). La console l'ignore. |
| `// deplace_x fait reculer la lettre et rend 0 ; « x = » range ce 0 dans x.` | Un commentaire : une phrase pour celui qui lit (« deplace_x fait reculer la lettre et rend 0 ; « x = » range ce 0 dans x. »). La console l'ignore. |
| `x = deplace_x(x, 0, ALPHABET[0], -pas);  // après cette ligne : x vaut 0` | x prend une nouvelle valeur : deplace_x(x, 0, ALPHABET[0], -pas). (Le commentaire le dit : « après cette ligne : x vaut 0 ».) |
| `while (true) {` | La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). |
| `image();  // la lettre reste en colonne 0` | Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « la lettre reste en colonne 0 ».) |
| `}` | L'accolade fermante : la fin du bloc ouvert plus haut par « { ». |
| `}` | L'accolade fermante : la fin du bloc ouvert plus haut par « { ». |

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1092 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        63 o  ← la console, à l'allumage  — la mise en route de la console
  deplace_x()     158 o  ← main()  — une fonction fournie par le langage
```

```
  main()           49 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()     × 2    26 o sur place, dans deplace_x()  + AdresseCase (10 o)
  effacer()   × 1    15 o sur place, dans deplace_x()  + AdresseCase (10 o), EffacerCases (12 o)
  image()     × 2     6 o sur place, dans deplace_x(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  deplace_x() × 2    38 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x>   // fait avancer une tuile sur sa ligne
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5;      // +5 : avance de 5 cases ; -5 : recule de 5 cases
```

3. UNE VARIABLE — X

```
uint8_t x = 0;        // la colonne de la lettre : elle part de 0 (tout à gauche)
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  // L'ALLER (le 0.18) : 5 cases vers la droite ; x reçoit 5.
  x = deplace_x(x, 0, ALPHABET[0], pas);    // après cette ligne : x vaut 5

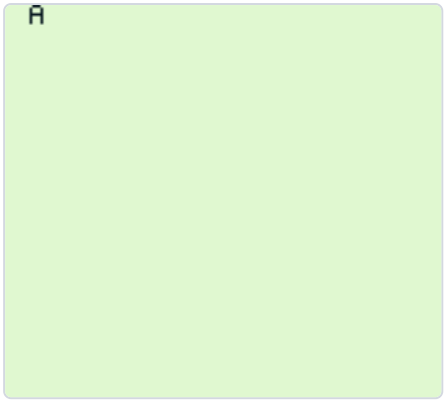
  // LE RETOUR, la ligne ajoutée :
  //
  //   x = deplace_x(x, 0, ALPHABET[0], -pas);
  //                 |                  |
  //                 |                  +-- -pas : -5, un pas NÉGATIF, donc vers
  //                 |                      la gauche. (Dans l'octet, -5 est rangé
  //                 |                      251 ; au-delà de 127, c'est un recul.)
  //                 +-- x vaut 5 grâce à l'aller : le retour part de la colonne 5
  //
  // deplace_x fait reculer la lettre et rend 0 ; « x = » range ce 0 dans x.
  x = deplace_x(x, 0, ALPHABET[0], -pas);   // après cette ligne : x vaut 0

  while (true) {
    image();          // la lettre reste en colonne 0
  }
}
```

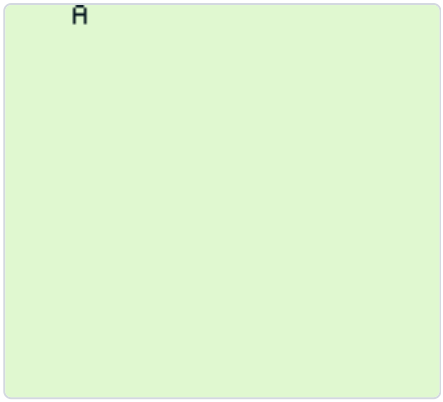
## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

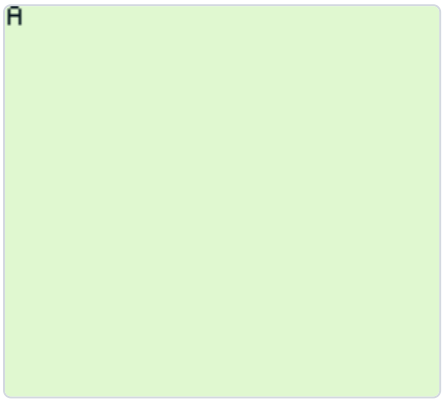
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d’une seconde — image 60



au bout de trois secondes — image 180

## 4. Ce qu’on doit voir

> Un A qui avance de 5 cases vers la droite, puis revient de 5 cases vers la gauche, et s’arrête.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d’être joués, et voici ce qu’ils ont mesuré.

- ✓ à l’aller, le A arrive en colonne 5 (« A »)
- ✓ au retour, le A revient en colonne 0 (« A »)
- ✓ x vaut la colonne rendue (x = 0)

---

Ensuite : **0.19.1. Revenir : un pas négatif — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++